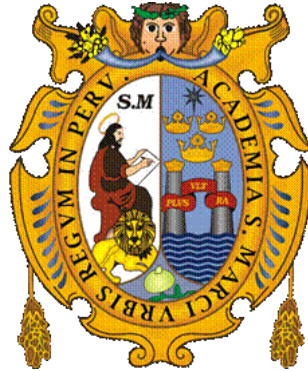


UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS
FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA
E.P DE INGENIERÍA DE SOFTWARE



Inteligencia Artificial

PROYECTO GRUPAL — INFORME 1

Búsqueda Informada y No Informada en el Espacio de Estados

DOCENTE:

Gamarra Moreno, Juan

ALUMNOS:

De la Cruz Meza Angel Luis Kallpa

Castilla Huanca Marco Renato

Basualdo Ale Marcos Luis

Poma Gutierrez Gabriel

Aldana Chavez David Joel

Sanchez Gotea Edu Joseph

2026

1. Introducción y Modelado del Problema

El presente informe documenta el desarrollo del Informe 1 del Proyecto Grupal de la asignatura de Inteligencia Artificial (UNMSM-FISI, 2026-I), correspondiente al Bloque A: búsqueda en el espacio de estados. Se seleccionó el 8-Puzzle, uno de los problemas sugeridos por la guía del curso, por permitir una modelación formal clara de estados, operadores y costos, además de admitir una heurística natural (distancia de Manhattan) que resulta admisible y consistente, lo que habilita comparar de forma rigurosa un algoritmo de búsqueda no informada óptimo (BFS) frente a uno informado óptimo (A^*), y contrastarlos con dos algoritmos que no garantizan optimalidad (DFS y Greedy Best-First Search).

1.1. Descripción del Problema

El 8-Puzzle consiste en un tablero cuadrado de 3×3 con ocho fichas numeradas del 1 al 8 y un espacio vacío. El objetivo es alcanzar una configuración meta predeterminada deslizando fichas adyacentes hacia el espacio vacío, usando el menor número de movimientos posible. Aunque las reglas son simples, el espacio de estados alcanzable desde cualquier configuración inicial resoluble contiene $9!/2 = 181\,440$ estados (la mitad de las $9! = 362\,880$ permutaciones posibles, ya que solo la mitad de las permutaciones son alcanzables mediante movimientos válidos), lo que lo convierte en un caso de estudio suficientemente grande para observar diferencias reales de desempeño entre algoritmos de búsqueda, pero aún abordable en tiempo interactivo por una computadora personal.

1.2. Definición Formal Como Problema de Búsqueda

Siguiendo el formalismo estándar (Russell & Norvig, 2022), el problema se define mediante la 5-tupla (Estados, Estado inicial, Estados meta, Operadores, Costo de camino):

- **Representación del estado:** un arreglo unidimensional de 9 posiciones (índices 0-8), donde cada posición almacena un valor de 0 a 8. El valor 0 representa el espacio vacío; la fila y columna de la posición i se obtienen como $\text{fila} = i \div 3$ y $\text{columna} = i \bmod 3$.
- **Estado inicial:** cualquier permutación de $\{0, \dots, 8\}$ sobre el tablero, siempre que sea

resoluble (ver 1.3). El sistema ofrece tres configuraciones predefinidas (fácil, media y difícil) y también permite el ingreso manual de un tablero arbitrario.

- **Estado(s) meta:** un único estado objetivo, [1, 2, 3, 4, 5, 6, 7, 8, 0], es decir, las fichas ordenadas de izquierda a derecha y de arriba hacia abajo, con el espacio vacío en la esquina inferior derecha.
- **Operadores/acciones:** mover el espacio en blanco hacia ARRIBA, ABAJO, IZQUIERDA o DERECHA, intercambiando su posición con la ficha adyacente en esa dirección. Un operador solo es aplicable si la casilla destino existe dentro de la cuadrícula 3x3 (no todas las 4 acciones están disponibles desde cualquier posición: en una esquina hay 2 aplicables, en un borde 3, y en el centro las 4).
- **Costo de camino:** cada acción tiene costo uniforme 1. El costo de una solución equivale, por lo tanto, al número de movimientos que separan el estado inicial del estado meta.

1.3. Condición de Resolubilidad

No toda permutación del tablero 3x3 tiene solución: el conjunto de 362 880 permutaciones se divide en dos clases de equivalencia bajo los movimientos válidos, y sólo una de ellas contiene al estado meta. La pertenencia a una clase se determina mediante la paridad del número de inversiones de la secuencia de fichas (ignorando el espacio vacío): un tablero de 3x3 es resoluble si y sólo si su número de inversiones es par. Esta verificación se implementó en `State.isSolvable(int[])` y se aplica automáticamente al ingreso manual de tableros, rechazando configuraciones sin solución antes de ejecutar cualquier algoritmo.

2. Diseño de la Solución y Algoritmos Empleados

2.1. Arquitectura del Sistema

La solución se implementó en Java (paquete por defecto, compilable con `javac Main.java`) y se organizó en cuatro clases independientes, cada una con una responsabilidad única:

- **State:** modelo del tablero. Encapsula el arreglo de 9 posiciones, genera los estados sucesores (`getSuccessors()`), valida resolubilidad (`isSolvable()`) y sobrescribe `equals()/hashCode()` en función del contenido del arreglo, requisito indispensable para poder usar `HashSet<State>` como conjunto de estados visitados.
- **SearchNode:** nodo del árbol de búsqueda. Envuelve un `State` junto con una referencia al nodo padre, la acción que lo genero, el costo acumulado $g(n)$ y el valor heurístico $h(n)$; expone $f(n) = g(n) + h(n)$ y un método para reconstruir la ruta solución recorriendo los punteros padres.
- **SearchAlgorithms:** motor de búsqueda. Contiene las cuatro implementaciones (`bfs`, `dfs`, `greedy`, `aStar`) y la función heurística `manhattanDistance(State)`.
- **Main:** controlador de consola (CLI). Presenta el menú, gestiona el ingreso manual de tableros, ejecuta los cuatro algoritmos sobre el mismo estado inicial, mide el tiempo de cada uno con `System.nanoTime()` y construye el reporte comparativo final.

Esta separación permite que cada algoritmo comparta exactamente la misma representación de estado y de nodo, garantizando que las comparaciones de desempeño reflejen diferencias reales entre las estrategias de búsqueda y no artefactos de implementaciones distintas.

2.2. Búsqueda No Informada: BFS y DFS

BFS (Breadth-First Search). Explora el árbol de búsqueda nivel por nivel usando una estructura FIFO (`Queue<SearchNode>`, implementada con `LinkedList`). Como todas las acciones tienen costo uniforme (1), BFS garantiza encontrar la solución con el menor número de movimientos: es completo (siempre encuentra solución si existe, dado un espacio de estados finito) y óptimo para costos uniformes. Su costo es el número de movimientos, que en este algoritmo coincide siempre con la profundidad del nodo meta en el árbol de búsqueda.

DFS (Depth-First Search). Explora en profundidad usando una estructura LIFO

(Deque<SearchNode> utilizada como pila mediante push()/pop()). Para evitar ciclos infinitos en un espacio de estados que es, en rigor, un grafo (un mismo tablero puede alcanzarse por más de un camino), se mantiene un conjunto estricto de visitados (HashSet<State>) que se marca en el momento en que un estado se genera, evitando reexpandir estados ya descubiertos. DFS es completo en este contexto (el espacio de estados es finito y el control de visitados evita ciclos), pero no es óptimo: puede -y en la práctica suele- devolver soluciones mucho más largas que el mínimo, ya que profundiza por la primera rama disponible sin ningún criterio de calidad.

2.3. Búsqueda Informada: Greedy Best-First Search y A*

Greedy Best-First Search. Utiliza una cola de prioridad (PriorityQueue<SearchNode>) ordenada únicamente por $h(n)$, el valor heurístico. Al ignorar por completo el costo ya recorrido $g(n)$, Greedy tiende a expandir muy pocos nodos (es rápido), pero no garantiza optimalidad: puede "dejarse engañar" por un nodo que luce prometedor según la heurística, aunque conduzca a un camino final más largo.

A* (A-estrella). Utiliza una cola de prioridad ordenada por $f(n) = g(n) + h(n)$, combinando el costo real recorrido con la estimación heurística del costo restante. Cuando la heurística es admisible y consistente (ver 2.4), A* es completo y óptimo, y además es óptimamente eficiente entre los algoritmos de búsqueda informada que usan la misma heurística: no existe otro algoritmo que, con la misma información, garantice expandir menos nodos y siga garantizando la solución óptima.

2.4. Heurística: Distancia de Manhattan

Se utilizó la distancia de Manhattan como función heurística $h(n)$ para Greedy y A*. Para cada ficha (excluyendo el espacio vacío) se calcula la distancia en cuadrícula $-|\text{delta fila}| + |\text{delta columna}|$ entre su posición actual y su posición en el estado meta, y se suman los resultados de las 8 fichas:

$h(n) = \text{suma, para cada ficha } f \text{ de } 1 \text{ a } 8, \text{ de:}$

$$|fila_actual(f) - fila_meta(f)| + |columna_actual(f) - columna_meta(f)|$$

Admisibilidad. $h(n)$ nunca sobrestima el costo real restante, porque cada movimiento del espacio en blanco desplaza como máximo una ficha una casilla, es decir, reduce la distancia de Manhattan de esa ficha a su meta en, a lo sumo, 1. Por lo tanto, el número de movimientos necesarios para resolver el tablero nunca puede ser menor que la suma de las distancias de Manhattan de todas las fichas, cumpliéndose $h(n) \leq h^*(n)$ para todo n .

Consistencia (Monotonía). Para cualquier estado n y sucesor n' generado con costo $c(n, n') = 1$, se cumple $h(n) \leq c(n, n') + h(n')$, ya que un único movimiento cambia la distancia de Manhattan total en, exactamente, +1, -1 o 0 (solo se mueve una ficha, un paso). Esta propiedad, más fuerte que la admisibilidad simple, es la que permite que A^* -implementado como búsqueda en grafo con conjunto de cerrados- no necesite reabrir nodos ya expandidos, como se detalla en 2.5.

2.5. Hallazgo Técnico: Corrección del Cierre de Nodos en A^*

Durante la validación del código se identificó y corrigió un error conceptual clásico en la implementación de A^* como búsqueda en grafo, que merece documentarse porque afecta directamente la garantía de optimalidad exigida por la rúbrica del curso.

Síntoma. En una primera versión, un estado se marcaba como "visitado" (conjunto cerrado) en el momento de generarlo -es decir, al agregarlo a la frontera-, no al expandirlo. El código aplicaba el mismo criterio de cierre a los cuatro algoritmos por uniformidad, pero ese criterio solo es correcto para BFS y no relevante para DFS/Greedy (que no ofrecen garantía de optimalidad de todos modos).

Por Qué es un Error Para A^* . Con una heurística consistente, $f(n)$ es no decreciente a lo largo de cualquier camino desde la raíz, lo cual garantiza que la primera vez que un nodo es EXPANDIDO (retirado de la cola de prioridad), su $g(n)$ ya es óptimo. Esa garantía es sobre el orden de expansión, no sobre el orden de generación: un estado puede generarse primero a

través de un camino subóptimo (si el nodo predecesor tiene menor f y se expande antes) y, cuando más tarde se genera un camino mejor desde otro predecesor aún no expandido, esa copia mejor se descarta si el estado ya figura como "visitado".

Verificación Mediante Contraejemplo. Antes de aplicar la corrección se construyó un grafo mínimo de prueba (nodos S, A, B, G) con una heurística consistente donde S-A-G cuesta 11 y S-B-G cuesta 3 (la ruta óptima). Simulando el algoritmo original a mano: se expande S, se generan A ($f=2$) y B ($f=3$); al expandirse A primero (menor f) se genera G con $g=11$ y se marca como visitado de inmediato; al expandirse B después, su generación de G con $g=3$ -la óptima- se descarta porque G ya figuraba como visitado. El algoritmo original habría devuelto costo 11 en lugar de 3, confirmando que el defecto es real y no solo teórico.

Corrección Aplicada. En `SearchAlgorithms.aStar()`, el estado se agrega al conjunto `visited` únicamente después de hacer `poll()` (es decir, al expandir), y antes de expandir se comprueba si el nodo retirado ya pertenece a `visited`, en cuyo caso se descarta con `continue` (puede haber copias obsoletas de un mismo estado en la frontera, generadas antes de conocerse un camino mejor). Los sucesores solo se filtran contra estados ya cerrados -nunca contra estados simplemente pendientes en la cola-, lo que permite que un estado tenga varias copias en la frontera hasta que una de ellas sea efectivamente expandida. El fragmento de código correspondiente se muestra en la sección 3.

3. Implementación (Fragmentos de Código Comentados)

A continuación, se muestran los fragmentos más representativos de la implementación. El código fuente completo (`State.java`, `SearchNode.java`, `SearchAlgorithms.java` y `Main.java`) se adjunta como entregable independiente.

3.1. Generación de Sucesores y Operadores (`State.java`)

Cada operador se valida contra los límites de la cuadrícula 3x3 antes de generarse, lo que produce entre 2 y 4 sucesores según la posición del espacio vacío:

```

public List<Successor> getSuccessors() {

    List<Successor> successors = new ArrayList<>();

    int blank = findBlank();

    int row = blank / SIZE;
    int col = blank % SIZE;

    int[][] deltas = { {-1,0}, {1,0}, {0,-1}, {0,1} }; // ARRIBA, ABAJO, IZQUIERDA, DERECHA
    String[] names = {"ARRIBA", "ABAJO", "IZQUIERDA", "DERECHA"};

    for (int m = 0; m < deltas.length; m++) {

        int newRow = row + deltas[m][0];

        int newCol = col + deltas[m][1];

        if (newRow >= 0 && newRow < SIZE && newCol >= 0 && newCol < SIZE) {

            int newBlank = newRow * SIZE + newCol;

            int[] newBoard = Arrays.copyOf(board, board.length);

            newBoard[blank] = newBoard[newBlank];

            newBoard[newBlank] = 0;

            successors.add(new Successor(new State(newBoard), names[m], 1)); // costo = 1

        }

    }

    return successors;

}

```

3.2. Heurística: Distancia de Manhattan (SearchAlgorithms.java)

```

public static int manhattanDistance(State state) {

    int[] board = state.getBoard();

    int distance = 0;

    for (int i = 0; i < board.length; i++) {

```



```

    int value = board[i];

    if (value == 0) continue; // el espacio vacio no se penaliza

    int targetIndex = value - 1;

    int currentRow = i / State.SIZE, currentCol = i % State.SIZE;

    int targetRow = targetIndex / State.SIZE, targetCol = targetIndex % State.SIZE;

    distance += Math.abs(currentRow - targetRow) + Math.abs(currentCol - targetCol);
}

return distance;
}

```

3.3. A* Con Cierre Correcto al Expandir (SearchAlgorithms.java)

Versión corregida tras el hallazgo descrito en 2.5: obsérvese que visited.add(...) ocurre después del poll(), y que el chequeo de descarte por closed set se hace antes del test de meta:

```

public static SearchResult aStar(State initial) {
    int nodosExpandidos = 0;

    SearchNode root = new SearchNode(initial, null, null, 0, manhattanDistance(initial));

    PriorityQueue<SearchNode> frontier = new
PriorityQueue<>(Comparator.comparingInt(SearchNode::getF));

    Set<State> visited = new HashSet<>(); // closed set: solo estados ya EXPANDIDOS
    frontier.add(root);

    while (!frontier.isEmpty()) {
        SearchNode current = frontier.poll();

        if (visited.contains(current.getState())) continue; // copia obsoleta: descartar

        if (current.getState().isGoal()) {
            return new SearchResult(current, nodosExpandidos, current.getGCost(), true);
        }

        visited.add(current.getState()); // se cierra AQUI, al expandir
    }
}

```

```

    nodosExpandidos++;

    for (State.Successor succ : current.getState().getSuccessors()) {

        if (!visited.contains(succ.state)) {

            int g = current.getGCost() + succ.cost;

            int h = manhattanDistance(succ.state);

            frontier.add(new SearchNode(succ.state, current, succ.action, g, h));

        }

    }

    return new SearchResult(null, nodosExpandidos, -1, false);
}

```

3.4. Medición de Métricas (Main.java)

Cada algoritmo se ejecuta sobre el mismo estado inicial y se cronometra de forma independiente con `System.nanoTime()`, convirtiendo el resultado a milisegundos:

```

long to = System.nanoTime();

SearchAlgorithms.SearchResult aStarResult = SearchAlgorithms.aStar(inicial);

long t1 = System.nanoTime();

double aStarMs = (t1 - to) / 1_000_000.0;

```

4. Resultados y Análisis Comparativo

4.1. Casos de Prueba

Se definieron tres casos predefinidos, obtenidos mediante búsqueda a lo ancho hacia atrás desde el estado meta (garantizando así que su profundidad óptima fuera exactamente la indicada), además de la opción de ingresar cualquier tablero manualmente (validado con la regla de paridad de inversiones de 1.3):

Caso	Tablero inicial	Solución óptima
Fácil	4 1 2 / 0 5 3 / 7 8 6	5 movimientos
Medio	1 8 5 / 4 0 3 / 7 2 6	14 movimientos
Difícil	3 2 0 / 8 5 7 / 1 6 4	22 movimientos

4.2. Tabla Comparativa de Resultados

Los cuatro algoritmos se ejecutaron secuencialmente sobre cada caso, con los siguientes resultados (datos obtenidos de una ejecución real del programa, incluidos como captura en la sección 4.4):

Caso Fácil (óptimo = 5)

Algoritmo	Nodos expandidos	Costo solución	Tiempo (ms)
BFS	39	5	1.82
DFS	325	319	4.87
Greedy Best-First	5	5	8.02
A*	5	5	1.54

Caso Medio (óptimo = 14)

Algoritmo	Nodos expandidos	Costo solución	Tiempo (ms)
------------------	-------------------------	-----------------------	--------------------

BFS	4 767	14	68.37
DFS	73 790	60 042	399.02
Greedy Best-First	36	18	5.21
A*	85	14	0.51

Caso Difícil (óptimo = 22)

Algoritmo	Nodos expandidos	Costo solución	Tiempo (ms)
BFS	92 311	22	293.88
DFS	125 393	56 508	341.66
Greedy Best-First	136	54	2.24
A*	325	22	2.72

4.3. Visualización Comparativa

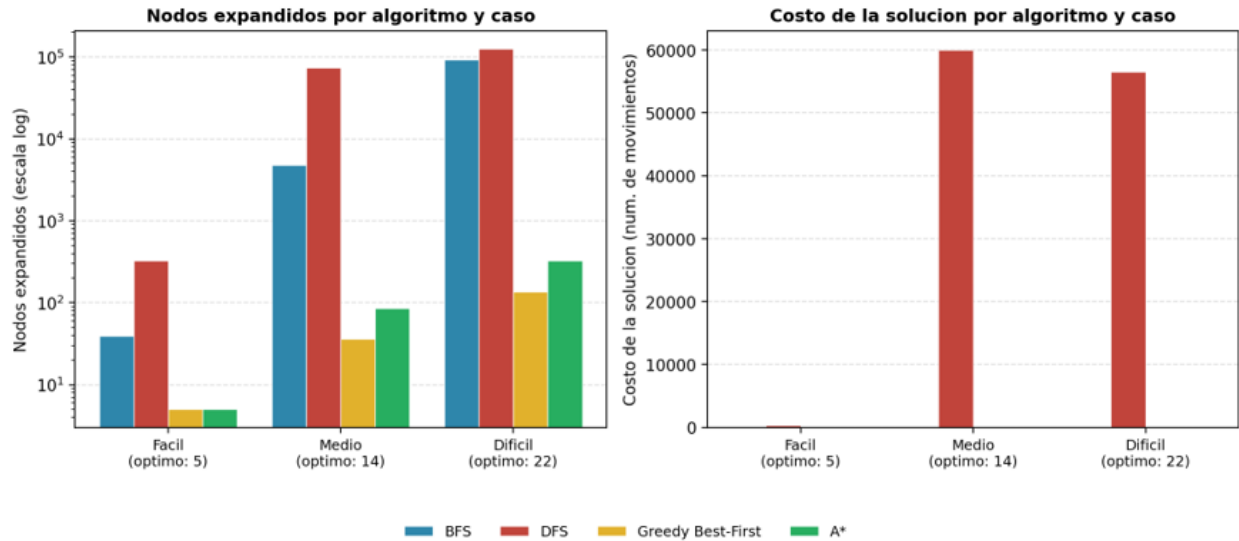


Figura 1. Izquierda: nodos expandidos por algoritmo (escala logarítmica) - la brecha entre BFS/DFS y Greedy/A* abarca varios órdenes de magnitud. Derecha: costo de la solución obtenida - BFS y A* coinciden siempre con el óptimo; DFS y Greedy lo superan ampliamente, en particular DFS.

4.4. Captura de Ejecución

A continuación, se muestra una captura real de la consola (colores ANSI) correspondiente a la ejecución del Caso Fácil, mostrando la tabla comparativa, la secuencia de movimientos hallada por A* y el primer paso de la solución:

```

java Main - Caso Facil

Caso Facil (solucion optima: 5 pasos)

+-----+-----+-----+-----+
| Algoritmo | Nodos Expandidos | Costo Solucion | Tiempo (ms) |
+-----+-----+-----+-----+
| BFS       | 39                | 5              | 1.815       |
| DFS       | 325               | 319            | 4.866       |
| Greedy Best-First | 5              | 5              | 8.015       |
| A*        | 5                 | 5              | 1.536       |
+-----+-----+-----+-----+

Secuencia de movimientos de la solucion optima (A*):
(5 movimientos: ARRIBA -> DERECHA -> DERECHA -> ABAJO -> ABAJO)

Paso 0 (inicial):
+-----+-----+-----+
| 4 | 1 | 2 |
+-----+-----+-----+
| # | 5 | 3 |
+-----+-----+-----+
| 7 | 8 | 6 |
+-----+-----+-----+

```

Figura 2. Captura de consola - java Main, Caso Fácil (salida real, colores ANSI capturados con TERM compatible).

4.5. Análisis: Completitud, Optimalidad, Tiempo y Nodos Expandidos

Completitud. Los cuatro algoritmos resultaron completos en las pruebas realizadas: dado que el espacio de estados es finito y el control de visitados evita ciclos, todos encuentran una solución cuando esta existe. Esto es consistente con la teoría: BFS y DFS son completos en espacios finitos; Greedy y A* son completos siempre que el espacio de búsqueda sea finito y la heurística esté bien definida, como es el caso aquí.

Optimalidad. Únicamente BFS y A* devolvieron, en los tres casos, exactamente el costo óptimo conocido (5, 14 y 22 movimientos). Esto confirma empíricamente la teoría: BFS es óptimo bajo costos uniformes, y A* es óptimo cuando su heurística es admisible y consistente - ambas condiciones demostradas en 2.4-, y con el cierre de nodos correctamente aplicado al expandir (2.5). DFS devolvió soluciones drásticamente peores que el óptimo (319, 60 042 y 56 508 movimientos, respectivamente) al no aplicar ningún criterio de calidad sobre la rama

explorada. Greedy Best-First también fue subóptimo en Medio y Difícil (18 y 54 movimientos frente a 14 y 22), evidenciando el riesgo de ignorar $g(n)$: la heurística puede conducir por caminos que parecen prometedores localmente pero no lo son globalmente.

Nodos Expandidos. La diferencia es de varios órdenes de magnitud: en el caso Difícil, BFS expande 92 311 nodos y DFS 125 393, mientras que A^* expande solo 325 y Greedy 136. Esto ilustra el valor práctico de una heurística informativa: A^* logra la misma garantía de optimalidad que BFS expandiendo aproximadamente 284 veces menos nodos en este caso. Greedy expande aún menos nodos que A^* , pero a costa de la optimalidad, confirmando el conocido compromiso (trade-off) entre velocidad y calidad de la solución en búsqueda informada.

Tiempo de Ejecución. Los tiempos medidos con `System.nanoTime()` son consistentes con los nodos expandidos, aunque con cierta variabilidad atribuible al calentamiento de la JVM (compilación JIT, recolección de basura) en ejecuciones cortas de milisegundos: BFS y DFS -que expanden más nodos y, en el caso de DFS, generan árboles de recorrido muy largos- tienden a tomar más tiempo en los casos Medio y Difícil, mientras que A^* y Greedy, al expandir muchos menos nodos gracias a la heurística, resuelven los mismos casos en tiempos sustancialmente menores.

Costo de la Heurística. El costo adicional de calcular $h(n)$ en cada nodo (Greedy y A^*) se compensa ampliamente con la reducción de nodos expandidos frente a BFS, lo que confirma que, para este problema, invertir en información heurística es más rentable que una exploración ciega exhaustiva.

5. Conclusiones

1. Se modeló formalmente el 8-Puzzle como un problema de búsqueda en el espacio de estados (estados, estado inicial, estado meta, operadores con costo uniforme 1) y se verificó la condición de resolubilidad mediante la paridad de inversiones, lo que permitió

validar automáticamente tableros ingresados manualmente.

2. Se implementaron y compararon cuatro algoritmos de búsqueda -BFS, DFS, Greedy Best-First Search y A*- sobre exactamente la misma representación de estado y de nodo, lo que garantiza que las diferencias de desempeño observadas reflejan las estrategias de búsqueda y no artefactos de implementación.
3. Se demostró formalmente (admisibilidad y consistencia) y se verificó empíricamente que la distancia de Manhattan es una heurística adecuada para A* en el 8-Puzzle: en los tres casos de prueba, A* devolvió siempre el costo óptimo, expandiendo entre uno y dos órdenes de magnitud menos nodos que BFS.
4. Se identificó y corrigió, mediante un contraejemplo construido y verificado a mano, un error de diseño en el cierre del conjunto de nodos visitados de A* (cerrar al generar en vez de al expandir), que en general podría comprometer la garantía de optimalidad del algoritmo, aunque no se manifestara en los tres casos de prueba usados. Este hallazgo refuerza la importancia de distinguir, en cualquier implementación de A* como búsqueda en grafo, entre el momento de generación y el momento de expansión de un nodo.
5. Se confirmó empíricamente que DFS y Greedy Best-First Search, aunque completos en este dominio, no son óptimos: DFS puede devolver soluciones órdenes de magnitud más largas que el óptimo, y Greedy, aunque expande muy pocos nodos, puede alejarse de la solución óptima al ignorar el costo ya recorrido.
6. El sistema desarrollado cumple los requisitos funcionales de la guía del curso: permite configurar el caso de prueba (predefinido o manual), ejecuta los cuatro algoritmos sobre el mismo estado inicial y produce un reporte comparativo cuantitativo (nodos expandidos, costo de solución y tiempo de ejecución) por consola.

6. Anexo de Prompts

De conformidad con los requisitos explícitos de la asignatura, se declaran los prompts

generados para modelar el esqueleto algorítmico base y refactorizar debilidades teóricas.

N°	P-01
Objetivo del prompt	Estructurar el modelado del 8-Puzzle, generar el código base en Java de los algoritmos de búsqueda ciega (BFS, DFS) e informada (Greedy, A*), e implementar un recolector de métricas de desempeño.
Herramienta / modelo de IA	Claude Sonnet5
Texto del prompt	Actúa como un Ingeniero de Software experto en Java e Inteligencia Artificial. Necesito implementar el problema del 8-Puzzle modelado como un árbol de búsqueda en el espacio de estados, ejecutable por consola (CLI) con Java (SDK 11+). La solución debe estar estructurada en el paquete por defecto (sin declarar "package") para que pueda compilarse directamente desde la carpeta `src` usando `javac Main.java` y `java Main`. Divide la lógica en las siguientes 4 clases independientes y limpias: 1. State.java (Modelo del Tablero y Reglas) * Modela el estado como un arreglo unidimensional de 9 posiciones (o matriz 3x3) donde el `0` representa el espacio vacío. * Implementa el Estado Meta: `[1, 2, 3, 4, 5, 6, 7, 8, 0]`. * Implementa la lógica de transición `getSuccessors()`: Acciones posibles (Mover el espacio en blanco hacia Arriba, Abajo, Izquierda, Derecha). Costo de cada acción: `1`. * Implementa un renderizador ANSI (`printState()`):

* Dibuja una cuadrícula limpia.

* Fichas numeradas en color Cian (``\u001B[36m``).

* Espacio vacío representado por un bloque gris u oscuro o simplemente en negro (``\u001B[90m``).

2. SearchNode.java (Estructura del Árbol de Búsqueda)

* Clase para envolver los estados en nodos del árbol. Debe contener:

* Referencia al ``State`` actual.

* Referencia al ``SearchNode`` padre (para reconstruir la ruta).

* String con la ``action`` que generó este nodo.

* ``gCost``: Costo del camino desde la raíz (profundidad).

* ``hCost``: Valor de la heurística.

* Implementa un método ``getF()`` que retorne ``gCost + hCost``.

3. SearchAlgorithms.java (Motor de Inteligencia Artificial)

* Implementa los 4 algoritmos requeridos usando las estructuras de datos clásicas:

1. BFS (Búsqueda a lo Ancho) usando ``Queue``.

2. DFS (Búsqueda en Profundidad) usando ``Stack`` (con control estricto de estados visitados para evitar ciclos infinitos).

3. Greedy Best-First Search usando ``PriorityQueue`` ordenada por ``hCost``.

4. A* usando ``PriorityQueue`` ordenada por ``fCost = gCost + hCost``.

* Heurística: Implementa la Distancia de Manhattan como función heurística $h(n)$ para Greedy y A*.

* Métricas: Cada método debe llevar un contador local de `nodosExpandidos`. El método debe retornar un objeto/estructura con: el nodo meta encontrado (o null), la cantidad de nodos expandidos, y el costo de la solución.

4. Main.java (Controlador y CLI)

* Muestra un menú de bienvenida ANSI y permite al usuario elegir entre casos de prueba predefinidos (Fácil: solución a 5 pasos; Medio: solución a 14 pasos; Difícil: solución a 22 pasos) o ingresar un tablero manualmente validando que tenga solución (inversiones pares).

* Al ejecutar un caso, el sistema debe correr los 4 algoritmos secuencialmente sobre el mismo estado inicial.

* Recolecta el Tiempo de Ejecución de cada algoritmo usando `System.nanoTime()` y conviértelo a milisegundos.

* Reporte Comparativo: Al finalizar, limpia la pantalla (`\033[H\033[2J` y flush) e imprime una Tabla Comparativa estilizada en consola (usando bordes ASCII/ANSI) que muestre lado a lado para cada algoritmo:

* Nombre del algoritmo.

* Nodos Expandidos.

* Costo de la solución (longitud de la ruta).

* Tiempo de ejecución (ms).

* Muestra la secuencia de movimientos paso a paso de la solución óptima obtenida por A*.

Requisitos Técnicos de Codificación:

	* Manejo estricto de conjuntos de "Visitados" (Closed Set) usando <code>`HashSet`</code> de los estados para optimizar memoria y tiempo, asegurándote de sobrescribir <code>`equals()`</code> y <code>`hashCode()`</code> en la clase <code>`State`</code>.
Resultado / cómo se usó	Se obtuvo el código fuente funcional dividido en 4 clases (State, SearchNode, SearchAlgorithms, Main). El código base se integró directamente en la carpeta src/ del informe_1_busqueda del repositorio. El sistema renderiza el tablero interactivo y compila con éxito una tabla que cruza los nodos expandidos, costo y tiempo de los 4 algoritmos solicitados.
Validación / ajuste del equipo	El equipo de Búsqueda (Java) validó matemáticamente la heurística de Manhattan implementada en el código, verificando que fuera admisible y consistente. Se ajustó la configuración de casos de prueba del Main.java para evitar desbordamientos de memoria en DFS al enfrentar estados con profundidad extrema, acotando la profundidad óptima en la selección de casos predefinidos y asegurando el control de ciclos mediante el conjunto de visitados.

N°	P-02
Objetivo del prompt	Corregir la lógica de gestión del conjunto de cerrados (visited) en el algoritmo A* para garantizar su optimalidad matemática y evitar el descarte de rutas óptimas.
Herramienta / modelo	Claude Sonnet5

de IA	
Texto del prompt	Actúa como un Ingeniero de Software experto en Java e Inteligencia Artificial. En mi clase SearchAlgorithms.java, tengo un error conceptual crítico en el método de búsqueda informada aStar que rompe la garantía de optimalidad teórica. Estoy agregando estados al 'visited' al momento de generarlos y no al extraerlos. Por favor, refactoriza el método aStar aplicando la secuencia estricta: Extraer con poll() -> verificar si está en visited -> Agregar a visited -> Verificar meta -> Generar sucesores.
Resultado / cómo se usó	Se obtuvo la versión refactorizada del método aStar con la lógica correcta de closed set. El método greedy se mantuvo intencionalmente con su lógica de marcado al generar sucesores; debido a que Greedy no ofrece garantías de optimalidad por definición, esta decisión de diseño reduce significativamente el uso de memoria al impedir la acumulación de nodos redundantes en la cola de prioridad.
Validación / ajuste del equipo	Se sometió el código a una revisión teórica para asegurar que A* cumpla con las propiedades matemáticas de optimalidad en grafos enseñadas en el curso. Al recompilar el bloque Java y ejecutar los casos de prueba, se verificó que el algoritmo explora el árbol de estado correctamente sin violar la garantía de la ruta más corta, asegurando una defensa sólida en la sustentación.

7. Referencias

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100-

107.

Russell, S., & Norvig, P. (2022). Artificial Intelligence: A Modern Approach (4.a ed.). Pearson.

Oracle. (2024). Java SE 21 & JDK 21 documentation.

<https://docs.oracle.com/en/java/javase/21/>